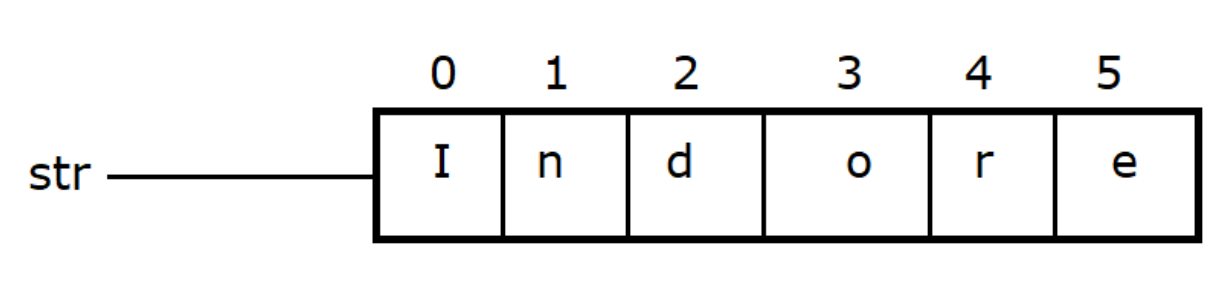
Working with methods of String class :

String str = "Indore";



String class has provided number of predefined methods to work with String which are as follows :-

1) public char charAt(int indexPosition) :-

It is used to extract a character based on the index position.

```
 CharAt.java
 -------------
 void main()
 {
     String str = "Indore";
     IO.println(str.charAt(0));
     IO.println(str.charAt(1));

     char ch =  str.charAt(2);
     IO.println(ch);
 }
```

public String concat(String str) :-

It is used to concatenate two Strings. This can be also done by using concatenation operator '+'.

```
Concat.java
--------------
void main()
{
    String s1 = "Data";
    String s2 = "base";

    String str = s1.concat(s2);
    IO.println(str);

    IO.println("Naresh".concat(" Technology"));

    String s3 = "Tata";
    String s4 = "Nagar";
   String s5 = s3 + s4;
    IO.println(s5);

}
```

IQ :

What is the difference between == operator and equals(Object obj) method?

equals(object obj) method of String class compares two strings based on the content because it is an overriden method where as == operator compares two Strings based on the reference i.e memory address.

```
IQ.java
---------
void main()
{
    String s1 = "Java";
    String s2 = new String("Java");
    IO.println(s1==s2); //false

    IO.println(s1.equals(s2)); //true
    IO.println(s1.hashCode()+" : "+s2.hashCode());

}
```

Note :- String class has overridden equals(Object obj) method from Object class because Object class equals(Object obj) method meant for memory address comparison but this overridden String class equals(Object obj) meant for content comparison.

Note :- String is a final class in java

public boolean equalsIgnoreCase(String str) :-

It is used to compare two Strings based on the content by ignoring the case.

```
IgnoreCase.java
------------------
void main(String [] args)
{
      String username = args[0];

        if(username.equalsIgnoreCase("rAvi"))
        {
            System.out.println("Welcome User!!!!");
        }
        else
        {
            System.out.println("Sorry! wrong username /Password");
        }

}
```

public String replace(char oldChar, char newChar)
public String replace(CharSequence target, CharSequence replacement)

It is an overloaded method of String class. The main purpose of this method to replace a character or a String OR StringBuffer OR StringBuilder object with another character or String OR StringBuffer OR StringBuilder object. The return type of this method is String.

CharSequenec is a predefined interface, String, StringBuffer and StringBuilder classes are implementing from CharSequence so we can assign String, StringBuffer and StringBilder object to CharSequence. (Loose Coupling)

```
Replace.java
---------------
void main()
{
  String s1 = "ababababab";
  IO.println("Original String :"+s1);

  String s2 = s1.replace('b','B');
  IO.println("After replacing :"+s2);

  String s3 = "Manager";
  IO.println(s3.replace("Man","Dam"));
}
```

---

public int compareTo(String s) :-

It is an overriden method from Comparable<T> interface because String class implements Comparable<T> interface.

It is used to compare two Strings based on character by character, comparison of two String Objects, chracter by chracter based on the UNICODE values are called Lexicographical comparison or dictionary comparison or alphabetical comparison(String case).

The return type of this method is int. It takes String as a parameter.

If s1 and s2 are two valid Strings

if s1==s2 -> 0

if s1>s2 -> +ve

if s1<s2   -> -ve

It has overloaded method i.e public int compareToIgnoreCase(String s)

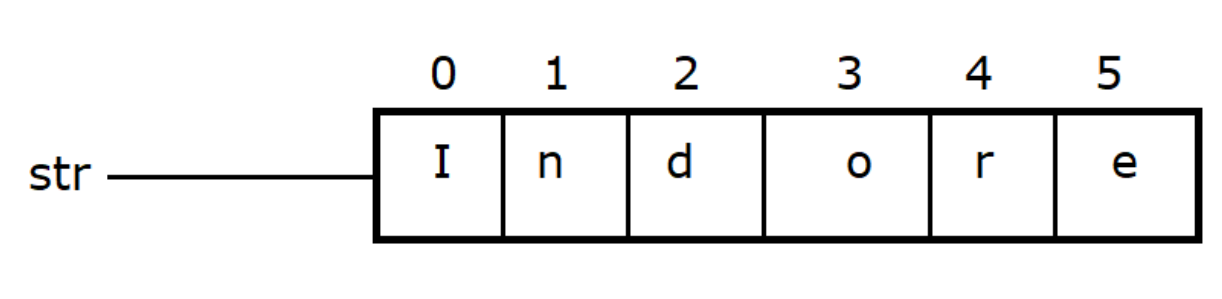
```
CompareTo.java
--------------------
void main()
{
  String s1 = "A";
  String s2 = "A";

  int compareValue = s1.compareTo(s2);  //65  -  65 = 0
  IO.println(compareValue);


  String s3 = "Sachin";
  String s4 = "Ratan";

  IO.println(s3.compareTo(s4));   //S >  R       //PQRS


  String s5 = "Raj";
  String s6 = "Rahul";
  IO.println(s6.compareTo(s5));    //j > h
}
```

---

public String substring(int startIndex)

public String substring(int startIndex, int endIndex)

String str = "Indore";



The main purpose of this method to extract the part of the existing String based on the index position.

startIndex is inclusive where as endIndex is exclusive.

if startIndex value is greater than endIndex then we will get an exception i.e StringIndexOutOfBoundsException.
substring(5,2);

If start index and end index both are equal, nothing will print

Nither start index nor end index will accept (-ve) value otherwise StringIndexOutOfBoundsException.

```
void main()
{
  String str = "IndoreCity";
  IO.println(str.substring(2));      //doreCity
  IO.println(str.substring(2,7));    //doreC
  IO.println(str.substring(4,4));    //Will not print anything
  //IO.println(str.substring(4,2));  //StringIndexOutOfBoundsException
  IO.println(str.substring(-3,8));    //StringIndexOutOfBoundsException
}
```

---

public boolean isEmpty() :-

It is used to verify whether a String is empty or not. This method returns true if the String is empty that means length is 0, otherwise it will return false.

```
void main()
{
      String str1 = "Java by James Gosling";
        String str2 = "";
        String str3 = " ";

        System.out.println(str1.isEmpty());
        System.out.println(str2.isEmpty());
        System.out.println(str3.isEmpty());
}
```

---

public boolean isBlank()

It is used to verify whether a String is empty or contains only whitespace characters.
It was introduced from JDK 11V.

```
 void main()
 {
      String s1 = "";
       String s2 = "   ";
       String s3 = "\n\t ";
       String s4 = "Hello";

       System.out.println(s1.isBlank()); // true
       System.out.println(s2.isBlank()); // true
       System.out.println(s3.isBlank()); // true
       System.out.println(s4.isBlank()); // false

 }
```